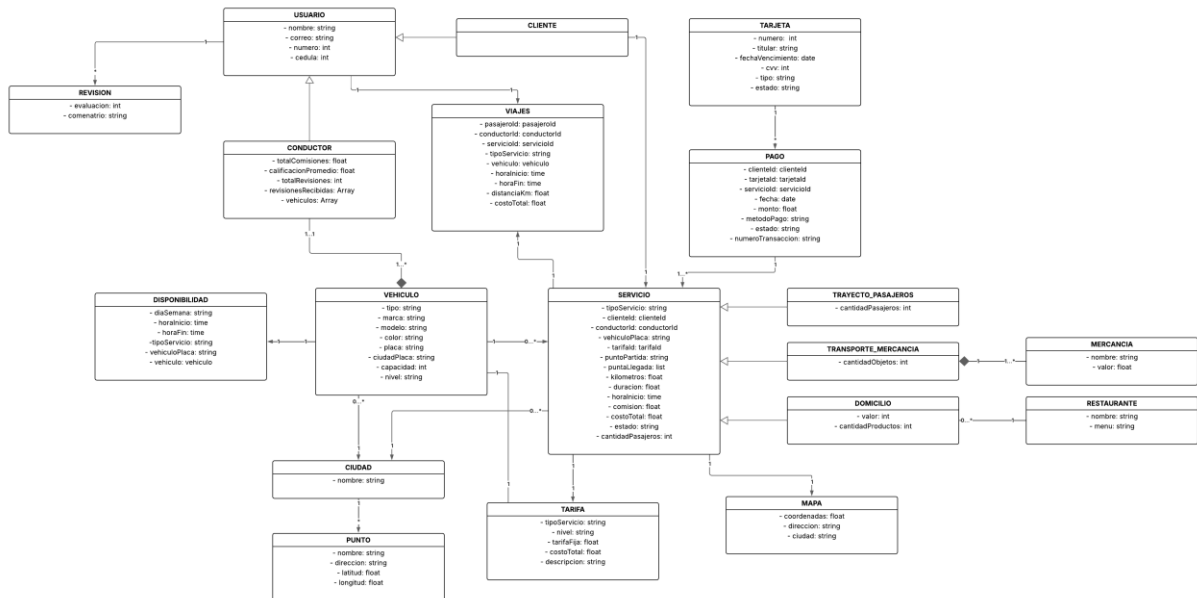


ENTREGA #3

Pregunta 1:

- a. Proponga un modelo conceptual en UML o E/R que describa las entidades del modelo de datos para la aplicación que se quiere desarrollar.



- b.** La lista de entidades con la descripción de cada una de ellas, las relaciones entre entidades y su cardinalidad (uno a uno, uno a muchos, o muchos a muchos).

Usuario:

- **Descripción:** Entidad base que almacena información común de todos los usuarios del sistema (tanto clientes como conductores).
- **Atributos:** nombre, correo, numero (celular), cedula
- **Relación con Revision:** Un usuario (sea cliente o conductor) puede dejar muchas revisiones (1:N)

Cliente:

- **Descripción:** Usuarios que solicitan servicios de transporte, comida o mercancías.
- **Atributos:** Hereda de Usuario
- **Relación con Usuario:** Herencia (1:1)
- **Relación con Servicio:** Un cliente puede solicitar muchos servicios (1:N)

Conductor:

- **Descripción:** Usuarios que prestan servicios usando sus vehículos registrados.
- **Atributos:** totalComisiones
- **Relación con Usuario:** Herencia (1:1)
- **Relación con Vehículo:** Un conductor puede tener muchos vehículos (1:N)
- **Relación con Servicio:** Un conductor presta muchos servicios (1:N)

ENTREGA #3

TarjetaCredito:

- **Descripción:** Medio de pago asociado a un cliente.
- **Atributos:** numero, nombreTarjeta, fechaVencimiento, codigo
- **Relación con Pago:** una tarjeta puede realizar múltiples pagos (1:N)

Pago:

- **Descripción:** Registro de transacciones realizadas por servicios.
- **Atributos:** monto, fecha, metodoPago, estado, numeroTransaccion
- **Relación con Servicio:** Un pago corresponde a un servicio (1:1)
- **Relación con TarjetaCredito:** Realizado con una tarjeta específica (N:1)

Vehículo:

- **Descripción:** Vehículos registrados por conductores para prestar servicios.
- **Atributos:** tipo, marca, modelo, color, placa, ciudadPlaca, capacidad, nivel
- **Relación con Conductor:** Pertenece a un conductor (N:1)
- **Relación con Actividad:** Un vehículo tiene una disponibilidad diaria (1:1)
- **Relación con Servicio:** Un vehículo presta muchos servicios (1:N)
- **Relación con Ciudad:** Registrado en una ciudad (N:1)

Actividad:

- **Descripción:** Disponibilidad horaria del conductor para prestar servicios.
- **Atributos:** dia, franjasHorarias
- **Relación con Vehículo:** Pertenece a un vehículo (1:1)

Servicio:

- **Descripción:** Entidad principal que representa cualquier servicio solicitado.
- **Atributos:** puntoPartida, puntoLlegada (lista), kilometros, duracion, horaInicio, horaFinalizacion, comision
- **Relación con Cliente:** Solicitado por un cliente (N:1)
- **Relación con Conductor:** Prestado por un conductor (N:1)
- **Relación con Vehículo:** Realizado con un vehículo (N:1)
- **Relación con Tarifa:** Usa una tarifa específica (N:1)
- **Relación con Mapa:** Tiene puntos de origen/destino (un mapa) (1:1)
- **Subtipos:** TrayectoPasajeros, Domicilio, TransporteMercancias

TrayectoPasajeros (Subtipo de Servicio):

- **Descripción:** Servicio de transporte de personas.
- **Atributos:** cantidadPasajeros

Domicilio (Subtipo de Servicio):

- **Descripción:** Servicio de entrega de comida.
- **Atributos:** valor, cantidadProductos
- **Relación con Restaurante:** Origen del pedido (N:1)

ENTREGA #3

TransporteMercancias (Subtipo de Servicio):

- **Descripción:** Servicio de transporte de objetos.
- **Atributos:** cantidadObjetos
- **Relación con Mercancia:** Transporta varias mercancías (1:N)

Restaurante:

- **Descripción:** Establecimientos asociados al servicio de domicilios.
- **Atributos:** nombre, menu
- **Relación con Domicilio:** Origen de muchos domicilios (1:N)

Mercancía:

- **Descripción:** Objetos transportados en servicios de mercancías.
- **Atributos:** nombre, valor
- **Relación con TransporteMercancias:** Parte de un transporte (N:1)

Tarifa:

- **Descripción:** Precio por kilómetro según tipo de servicio y nivel.
- **Atributos:** tarifaFija, costoTotal
- **Relación con Servicio:** Aplicada a muchos servicios (1:N)

Mapa:

- **Descripción:** Ubicaciones geográficas (puntos de partida y llegada).
- **Atributos:** coordenadas, direccion, ciudad
- **Relación con Servicio:** Un mapa para cada servicio (1:1)

Punto:

- **Descripción:** Ubicación específica dentro de una ciudad.
- **Atributos:** nombre, direccion, latitud, longitud
- **Relación con Ciudad:** Pertenece a una ciudad (N:1)

Ciudad:

- **Descripción:** Ciudades donde opera ALPESCAB.
- **Atributos:** nombre
- **Relación con Vehiculo:** Tiene vehículos registrados (1:N)
- **Relación con Punto:** Contiene muchos puntos (1:N)

Revision:

- **Descripción:** Calificaciones y comentarios entre usuarios.
- **Atributos:** evaluacion (0-5), comentario
- **Relación con Cliente:** Hecha por un cliente (N:1)
- **Relación con Conductor:** Sobre un conductor (N:1)

ENTREGA #3

Historial:

- **Descripción:** Registro histórico de servicios completados.
- **Relación con Servicio:** Agrupa servicios (1:N)

Punto:

- **Descripción:** Ubicación específica con coordenadas geográficas.
- **Atributos:** nombre, direccion, latitud, longitud
- **Relación con Servicio:** Un servicio tiene múltiples puntos (partida y llegadas) (N:1)
- **Relación con Ciudad:** Pertenece a una ciudad (N:1)

- c. El análisis de selección de esquema de asociación (modelo normalizado o embebido) para cada relación entre entidades, justificado a partir de los requerimientos de la aplicación y un análisis de la carga de trabajo descritas antes.

1. Usuario → Cliente/Conductor (Especialización 1:1)

Decisión: EMBEBIDO

Justificación:

- Siempre que consultamos un cliente necesitamos sus datos de usuario base
- La relación es 1:1 (un cliente ES un usuario)
- No tiene sentido consultar "Usuario" sin saber si es cliente o conductor
- En MongoDB no existe herencia real, entonces embebemos los campos de Usuario dentro de Cliente o Conductor

Análisis de carga:

- **Lecturas:** Alta frecuencia al consultar servicios o disponibilidad (siempre necesitamos nombre, correo, etc.)
- **Escrituras:** Baja frecuencia (usuarios actualizan datos cada 6 meses - 1 año)

3. Conductor → Vehículo (1:N)

Decisión: EMBEBIDO

Justificación:

- Aunque es 1:N, el promedio es 1.1 vehículos por conductor (muy pocos)
- Siempre que busquemos conductores disponibles, necesitamos info del vehículo
- Los vehículos no se consultan independientemente del conductor
- Actualización baja (cada 6 meses)

Análisis de carga:

ENTREGA #3

- **Lecturas:** Altísima frecuencia (búsqueda de conductores cada minuto)
- **Escrituras:** Baja (actualizaciones semestrales)

4. Vehículo → Actividad (1:N)

Decisión: EMBEBIDO

Justificación:

- Las disponibilidades siempre se consultan con el vehículo
- No tiene sentido consultar una "Actividad" sin saber de qué vehículo es
- Son pocas actividades por vehículo (días + franjas horarias)
- Actualización mensual pero siempre en conjunto
- Esencial para RF4 (consultar disponibilidad)

Análisis de carga:

- **Lecturas:** Altísima (cada búsqueda de conductor disponible)
- **Escrituras:** Media (actualizaciones mensuales)

5. Servicio → Cliente/Conductor/Vehículo (N:1)

Decisión: REFERENCIADO

Justificación:

- Un conductor tiene muchos de servicios (7-8 diarios $\times$ 365 días = 2,555 aprox servicios/año)
- Embeber servicios en conductor haría documentos gigantes (>16MB en poco tiempo)
- Los servicios se consultan de forma independiente (RFC1: histórico de usuario)
- Se necesita filtrar servicios por fecha, ciudad, tipo
- Los servicios se actualizan independientemente del conductor

Análisis de carga:

- **Escrituras:** Muy alta frecuencia (70,000 servicios diarios con 100k conductores $\times$ 70%)
- **Lecturas:** Alta al consultar históricos específicos

6. Servicio → Mapa (Puntos) (1:N)

Decisión: EMBEBIDO

Justificación:

ENTREGA #3

- Los puntos de partida/llegada son específicos de cada servicio
- Siempre se consultan junto con el servicio (RFC1, RF7)
- Son pocos (1 partida + 1-3 llegadas promedio)
- No tiene sentido consultar un "Mapa" sin el servicio asociado

Análisis de carga:

- **Lecturas:** Alta (cada consulta de histórico necesita los puntos)
- **Escrituras:** Alta (cada servicio nuevo)

7. Servicio → Tarifa (N:1)

Decisión: REFERENCIADO

Justificación:

- Las tarifas son catálogo compartido por muchos servicios
- Se actualizan centralmente (cambios de precio afectan a todos)
- Son pocas tarifas totales ($3 \text{ niveles} \times 3 \text{ tipos servicio} = \sim 9 \text{ tarifas}$)
- Se consultan para calcular precios antes de crear servicios
- Normalización necesaria para consistencia de precios

Análisis de carga:

- **Lecturas:** Alta (cada solicitud de servicio RF6)
- **Escrituras:** Muy baja

Análisis de carga:

- **Lecturas:** Alta (cada solicitud de servicio RF6)
- **Escrituras:** Muy baja (cambios de tarifa son esporádicos)

8. Domicilio → Restaurante (N:1)

Decisión: REFERENCIADO

Justificación:

- Los restaurantes son entidades independientes consultadas por usuarios
- Un restaurante tiene muchos domicilios asociados
- Los datos del restaurante (menú) se actualizan independientemente
- No todos los servicios son domicilios

Análisis de carga:

ENTREGA #3

- **Lecturas:** Alta (usuarios buscan restaurantes, consultan menús)
- **Escrituras:** Media (actualización de menús, horarios)

9. TransporteMercancias → Mercancia (1:N)

Decisión: EMBEBIDO

Justificación:

- Las mercancías son específicas de cada transporte
- No se consultan independientemente
- Son pocos objetos por envío (promedio 2-5)
- Se registran y consultan siempre con el servicio
- No se reutilizan entre servicios diferentes

Análisis de carga:

- **Lecturas:** Media (al consultar detalles del servicio)
- **Escrituras:** Media (cada servicio de mercancías)

10. Usuario ↔ Revision (N:M bidireccional)

Decisión: EMBEBIDO en Usuario

Justificación:

- Cada revisión pertenece a un servicio específico
- Las revisiones se consultan junto con el perfil del usuario
- Son relativamente pocas por usuario (no todos dejan revisión)
- Se necesita calcular promedio de calificaciones frecuentemente
- No se consultan revisiones de forma independiente

Análisis de carga:

- **Lecturas:** Alta (mostrar calificación promedio en cada búsqueda)
- **Escrituras:** Media (33% de servicios genera revisión según patrón típico)

11. Punto → Ciudad (N:1)

Decisión: EMBEBIDO

Justificación:

- Solo necesitamos el nombre de la ciudad en el punto
- No necesitamos toda la información de Ciudad cada vez

ENTREGA #3

- Las ciudades son catálogo pequeño (~50 ciudades operativas)
- Reduce joins innecesarios

Análisis de carga:

- **Lecturas:** Muy alta (cada servicio, cada búsqueda)
- **Escrituras:** Casi nula (ciudades no cambian)

12. Servicio → Historial (N:1)

Decisión: REFERENCIADO (implícito)

Justificación:

- El historial es solo una vista/consulta de servicios completados
- No es una colección física separada

13. Servicio → Punto (1:N - partida y llegadas)

Decisión: EMBEBIDO

Justificación:

- Los puntos de partida/llegada son específicos de cada servicio
- Siempre se consultan junto con el servicio (RFC1, RF7)
- Son pocos (1 partida + 1-3 llegadas promedio)
- No tiene sentido consultar un "Punto" sin el servicio asociado
- Las coordenadas capturadas son históricas del momento del servicio
- Crítico para cálculo de distancias y tarifas

Análisis de carga:

- **Lecturas:** Alta (cada consulta de histórico necesita los puntos)
- **Escrituras:** Muy alta (cada servicio nuevo registra puntos únicos)

14. Servicio (Especialización) → TrayectoPasajeros / Domicilio / TransporteMercancias

Decisión: EMBEBIDO con discriminador

Justificación:

- Los campos específicos de cada tipo se incluyen en el mismo documento
- Evita tener 3 colecciones separadas que complicarían las consultas
- Los atributos específicos son pocos (2-3 campos adicionales por tipo)

ENTREGA #3

Análisis de carga:

- **Lecturas:** Alta (RFC1, RFC2, RFC3 consultan todos los tipos de servicio)
- **Escrituras:** Alta (7-8 servicios diarios por conductor)

15. TransporteMercancias → Mercancia (1:N)

Decisión: EMBEBIDO (ya explicado, pero complemento)

Justificación:

- Las mercancías son **dependientes existencialmente** del servicio
- No existen mercancías sin un transporte asociado
- La información de mercancía es para registro histórico, no para consultas independientes
- Los usuarios no buscan "mercancías transportadas", buscan "mis servicios de transporte"

Análisis de carga:

- **Lecturas:** Media-Baja (solo al consultar detalle específico del servicio)
- **Escrituras:** Media (solo servicios de tipo TransporteMercancias)

16. Domicilio → Restaurante (N:1)

Decisión: REFERENCIADO

Justificación:

- Los usuarios consultan restaurantes ANTES de solicitar servicio (para ver menú, horarios)
- Un restaurante exitoso puede tener 100-200 domicilios por día
- Los datos del restaurante cambian frecuentemente (menú, horarios, promociones)

Análisis de carga:

- **Lecturas restaurante independiente:** Muy alta (usuarios explorando opciones)
- **Lecturas en contexto de domicilio:** Media (al ver histórico de pedidos)
- **Escrituras restaurante:** Alta (actualizaciones de menú, estado)

18. Servicio → Vehiculo (N:1)

Decisión: REFERENCIADO (por placa)

Justificación:

- Los servicios NO embeben todo el vehículo, solo guardan referencia por **placa**

ENTREGA #3

- El vehículo está embebido en Conductor, no es colección independiente
- La placa es suficiente para identificar el vehículo en históricos
- Si se necesita info completa del vehículo, se busca en el documento del conductor

Análisis de carga:

- **Lecturas servicio solo:** Alta (históricos RFC1)
- **Lecturas con info vehículo completa:** Media-Baja (detalles específicos)
- **Escrituras:** Alta (cada servicio nuevo)

19. Punto → Ciudad (N:1)

Decisión: EMBEBIDO (solo nombre de ciudad como string)

Justificación:

- Solo necesitamos el nombre de la ciudad en cada punto
- No necesitamos toda la información de Ciudad cada vez
- Las ciudades son catálogo pequeño (~50 ciudades operativas)
- Reduce joins innecesarios
- Suficiente para RFC3 (estadísticas por ciudad)

Análisis de carga:

- **Lecturas:** Muy alta (cada servicio, cada búsqueda)
- **Escrituras:** Casi nula (ciudades no cambian)

- d. Una descripción gráfica usando Json de cada relación entre entidades en donde presente un ejemplo de datos junto con el esquema de asociación usado (referenciado o embebido). En los anexos se muestra un ejemplo de lo que se requiere.

1. Conductor con Vehiculos y Actividades

```
{
  "_id": ObjectId("674a1234567890abcdef0001"),
  "tipo": "conductor",
  "nombre": "Carlos Mendez",
  "correo": "carlos.mendez@email.com",
  "numero": "3201234567",
  "cedula": "1234567890",
  "totalComisiones": 2450000.50,
  "vehiculos": [
    {
      "tipo": "carro",
      "marca": "Mazda",
      "modelo": "CX-5 2022",
      "color": "Negro",
```

ENTREGA #3

```
"placa": "ABC123",
"ciudadPlaca": "Bogotá",
"capacidad": 4,
"nivel": "Confort",
"actividades": [
  {
    "dia": "Lunes",
    "franjasHorarias": "06:00-12:00"
  },
  {
    "dia": "Lunes",
    "franjasHorarias": "14:00-20:00"
  },
  {
    "dia": "Martes",
    "franjasHorarias": "06:00-12:00"
  }
]
},
{
  "tipo": "motocicleta",
  "marca": "Yamaha",
  "modelo": "MT-03 2021",
  "color": "Azul",
  "placa": "XYZ789",
  "ciudadPlaca": "Bogotá",
  "capacidad": 1,
  "nivel": "Estándar",
  "actividades": [
    {
      "dia": "Sábado",
      "franjasHorarias": "08:00-18:00"
    },
    {
      "dia": "Domingo",
      "franjasHorarias": "08:00-18:00"
    }
  ]
}
]
```

2. Servicio TrayectoPasajeros

```
{
  "_id": ObjectId("674a1234567890abcdef0100"),
  "tipoServicio": "TrayectoPasajeros",
  "clienteId": ObjectId("674a1234567890abcdef0002"),
  "conductorId": ObjectId("674a1234567890abcdef0001"),
```

ENTREGA #3

```
"vehiculoPlaca": "ABC123",  
"tarifaId": ObjectId("674a1234567890abcdef0300"),
```

```
"puntoPartida": {  
  "nombre": "Centro Comercial Andino",  
  "direccion": "Carrera 11 # 82-71",  
  "ciudad": "Bogotá",  
  "coordenadas": {  
    "latitud": 4.6713,  
    "longitud": -74.0536  
  }  
},
```

```
"puntosLlegada": [  
  {  
    "nombre": "Aeropuerto El Dorado",  
    "direccion": "Calle 26 # 103-09",  
    "ciudad": "Bogotá",  
    "coordenadas": {  
      "latitud": 4.7016,  
      "longitud": -74.1469  
    }  
  }  
],
```

```
"kilometros": 15.3,  
"duracion": 35,  
"horaInicio": "2025-11-15T09:00:00Z",  
"horaFinalizacion": "2025-11-15T09:35:00Z",  
"comision": 9180.0,  
"costoTotal": 15300.0,  
"estado": "completado",
```

```
"cantidadPasajeros": 2  
}
```

3. Servicio Domicilio

```
{  
  "_id": ObjectId("674a1234567890abcdef0101"),  
  "tipoServicio": "Domicilio",  
  "clienteId": ObjectId("674a1234567890abcdef0002"),  
  "conductorId": ObjectId("674a1234567890abcdef0001"),  
  "vehiculoPlaca": "XYZ789",  
  "tarifaId": ObjectId("674a1234567890abcdef0301"),  
  "restauranteId": ObjectId("674a1234567890abcdef0400"),
```

```
"puntoPartida": {  
  "nombre": "Crepes & Waffles Usaquéen",
```

ENTREGA #3

```
"direccion": "Carrera 6 # 119-25",
"ciudad": "Bogotá",
"coordenadas": {
  "latitud": 4.6959,
  "longitud": -74.0301
}
},

"puntosLlegada": [
  {
    "nombre": "Apartamento cliente",
    "direccion": "Calle 127 # 7-12 Apto 503",
    "ciudad": "Bogotá",
    "coordenadas": {
      "latitud": 4.7123,
      "longitud": -74.0315
    }
  }
]

"kilometros": 2.8,
"duracion": 15,
"horaInicio": "2025-11-15T19:30:00Z",
"horaFinalizacion": "2025-11-15T19:45:00Z",
"comision": 1680.0,
"costoTotal": 2800.0,
"estado": "completado",

"valor": 45000,
"cantidadProductos": 3
}
```

4. Servicio TransporteMercancias con Mercancías

```
{
  "_id": ObjectId("674a1234567890abcdef0102"),
  "tipoServicio": "TransporteMercancias",
  "clienteId": ObjectId("674a1234567890abcdef0002"),
  "conductorId": ObjectId("674a1234567890abcdef0001"),
  "vehiculoPlaca": "ABC123",
  "tarifaId": ObjectId("674a1234567890abcdef0302"),

  "puntoPartida": {
    "nombre": "Bodega Principal",
    "direccion": "Calle 80 # 98-20",
    "ciudad": "Bogotá",
    "coordenadas": {
      "latitud": 4.6892,
      "longitud": -74.0894
    }
  }
}
```

ENTREGA #3

```
}  
},  
  
"puntosLlegada": [  
  {  
    "nombre": "Sucursal Norte",  
    "direccion": "Calle 170 # 7-34",  
    "ciudad": "Bogotá",  
    "coordenadas": {  
      "latitud": 4.7552,  
      "longitud": -74.0432  
    }  
  },  
  {  
    "nombre": "Sucursal Sur",  
    "direccion": "Autopista Sur # 45-12",  
    "ciudad": "Bogotá",  
    "coordenadas": {  
      "latitud": 4.5732,  
      "longitud": -74.1284  
    }  
  }  
],  
  
"kilometros": 28.5,  
"duracion": 65,  
"horaInicio": "2025-11-15T07:00:00Z",  
"horaFinalizacion": "2025-11-15T08:05:00Z",  
"comision": 17100.0,  
"costoTotal": 28500.0,  
"estado": "completado",  
  
"cantidadObjetos": 4,  
"mercancias": [  
  {  
    "nombre": "Caja de repuestos A",  
    "valor": 250000  
  },  
  {  
    "nombre": "Caja de repuestos B",  
    "valor": 180000  
  },  
  {  
    "nombre": "Herramientas",  
    "valor": 120000  
  },  
  {  
    "nombre": "Manuales técnicos",
```

ENTREGA #3

```
"valor": 50000
}
]
}
```

5. Restaurante

```
{
  "_id": ObjectId("674a1234567890abcdef0400"),
  "nombre": "Crepes & Waffles Usaquéen",
  "menu": "https://crepesywaffles.com.co/menu",
  "ubicacion": {
    "direccion": "Carrera 6 # 119-25",
    "ciudad": "Bogotá",
    "coordenadas": {
      "latitud": 4.6959,
      "longitud": -74.0301
    }
  },
  "horarioAtencion": "10:00-22:00",
  "telefono": "6013456789"
}
```

6. Tarifa

```
{
  "_id": ObjectId("674a1234567890abcdef0300"),
  "tipoServicio": "TrayectoPasajeros",
  "nivel": "Confort",
  "tarifaFija": 1000,
  "descripcion": "Tarifa por km para
}
```

7. Cliente

```
{
  "_id": ObjectId("674a1234567890abcdef0002"),
  "tipo": "cliente",
```

// Datos básicos heredados de Usuario

```
"nombre": "Ana Rodríguez",
"correo": "ana.rodriguez@email.com",
"numero": "3109876543",
"cedula": "9876543210",
```

// Sistema de calificaciones (para que conductores vean si aceptan al cliente)

```
"calificacionPromedio": 4.8,
"totalRevisiones": 15,
```

// Revisiones que ha RECIBIDO de conductores

```
"revisionesRecibidas": [
```

ENTREGA #3

```
{
  "servicioId": ObjectId("674a1234567890abcdef0100"),
  "conductorId": ObjectId("674a1234567890abcdef0001"),
  "evaluacion": 5,
  "comentario": "Pasajera muy amable y respetuosa, puntual",
  "fecha": "2025-11-15T10:35:00Z"
},
{
  "servicioId": ObjectId("674a1234567890abcdef0105"),
  "conductorId": ObjectId("674a1234567890abcdef0007"),
  "evaluacion": 5,
  "comentario": "Excelente cliente, muy educada",
  "fecha": "2025-11-18T14:20:00Z"
},
{
  "servicioId": ObjectId("674a1234567890abcdef0108"),
  "conductorId": ObjectId("674a1234567890abcdef0012"),
  "evaluacion": 4,
  "comentario": "Buena pasajera",
  "fecha": "2025-11-22T09:10:00Z"
}
],
}
```

8. Conductor

```
{
  "_id": ObjectId("674a1234567890abcdef0001"),
  "tipo": "conductor",

  // Datos básicos heredados de Usuario
  "nombre": "Carlos Méndez",
  "correo": "carlos.mendez@email.com",
  "numero": "3201234567",
  "cedula": "1234567890",

  // Específico de Conductor
  "totalComisiones": 2450000.50,

  // Sistema de calificaciones (para que clientes vean antes de aceptar)
  "calificacionPromedio": 4.7,
  "totalRevisiones": 234,

  // Revisiones que ha RECIBIDO de clientes
  "revisionesRecibidas": [
    {
      "servicioId": ObjectId("674a1234567890abcdef0100"),
      "clienteId": ObjectId("674a1234567890abcdef0002"),
      "evaluacion": 5,

```

ENTREGA #3

```
"comentario": "Excelente conductor, muy puntual y amable. Vehículo limpio.",
"fecha": "2025-11-15T10:30:00Z"
},
{
  "servicioId": ObjectId("674a1234567890abcdef0105"),
  "clienteId": ObjectId("674a1234567890abcdef0008"),
  "evaluacion": 4,
  "comentario": "Buen servicio, pero llegó 5 minutos tarde",
  "fecha": "2025-11-20T15:45:00Z"
},
{
  "servicioId": ObjectId("674a1234567890abcdef0112"),
  "clienteId": ObjectId("674a1234567890abcdef0015"),
  "evaluacion": 5,
  "comentario": "Muy profesional, excelente conductor",
  "fecha": "2025-11-25T18:30:00Z"
}
],
```

// Vehículos registrados (EMBEBIDO - relación 1:N)

```
"vehiculos": [
  {
    "tipo": "carro",
    "marca": "Mazda",
    "modelo": "CX-5 2022",
    "color": "Negro",
    "placa": "ABC123",
    "ciudadPlaca": "Bogotá",
    "capacidad": 4,
    "nivel": "Confort",
```

// Disponibilidades del vehículo (EMBEBIDO - relación 1:N)

```
"actividades": [
  {
    "dia": "Lunes",
    "franjasHorarias": "06:00-12:00",
    "tipoServicio": ["TrayectoPasajeros", "TransporteMercancias"]
  },
  {
    "dia": "Lunes",
    "franjasHorarias": "14:00-20:00",
    "tipoServicio": ["TrayectoPasajeros", "TransporteMercancias"]
  },
  {
    "dia": "Martes",
    "franjasHorarias": "06:00-12:00",
    "tipoServicio": ["TrayectoPasajeros"]
  },
],
```

ENTREGA #3

```
{
  "dia": "Miércoles",
  "franjasHorarias": "06:00-12:00",
  "tipoServicio": ["TrayectoPasajeros"]
},
{
  "dia": "Jueves",
  "franjasHorarias": "06:00-12:00",
  "tipoServicio": ["TrayectoPasajeros", "TransporteMercancias"]
},
{
  "dia": "Viernes",
  "franjasHorarias": "14:00-22:00",
  "tipoServicio": ["TrayectoPasajeros"]
}
]
}
]
```

Pregunta 3:

A continuación, se presenta el planteamiento de los casos de prueba con las precondiciones, los pasos de la prueba y los resultados esperados:

- RF1: las precondiciones serían verificar que no existan dos clientes con el mismo correo ni la misma cédula. Primero, se hace un POST de un cliente, luego se verifica la respuesta del servidor, y finalmente, se consulta con la colección de los clientes. La respuesta esperada es un código 201 de HTTP mostrando un éxito en el POST y que quedó registrado en la base de datos.
- RF2: las precondiciones serían verificar que no existe otro conductor con la misma cédula. Primero, se envía un POST a conductores, se verifica la respuesta del servidor y finalmente se consulta la colección de conductores. El resultado esperado es un nuevo documento de tipo conductor con los campos totalComisiones, totalRevisiones y calificacionPromedio con valores predeterminados y que los arreglos vehiculos y revisionesRecibidas esten vacios.
- RF3: Primero se debe verificar que el conductor existe. Se hace un POST a /conductores/{id}/vehiculos y luego se verifica el documento del conductor. Se espera que el arreglo vehiculos del conductor contenga al vehículo añadido y que no existan dos vehículos con las mismas placas.
- RF4: Se debe verificar que el conductor con el vehículo registrado existe. Se hace un POST a disponibilidades y luego se valida el documento en la colección. Se espera que el registro haya sido creado correctamente.
- RF5: Antes de iniciar la prueba se debe verificar que existe una disponibilidad con el id especificado y que no se sobreponga con otros horarios. La prueba consiste en enviar un PUT a /disponibilidades/{id} y luego consultar la colección para verificar los cambios. Se espera que la disponibilidad se actualice y que no se cruce con otros horarios.
- RF6: Se debe verificar que el cliente exista, que hay disponibilidades activas y que hay tarifas configuradas. La prueba empieza enviando un POST a servicios y se verifica la respuesta del servidor. Se espera la creación de un documento en servicios, la asignación entre conductorId y vehiculoPlaca, el costo de costoTotal, comision y duracion, y que el estado inicial muestre que inició el viaje.
- RF7: Se debe verificar que un servicio en curso exista. Se envía un PUT a /servicios/{id}/finalizar y verificar los datos actualizados. Se espera que el estado del servicio sea completado, que se actualicen horaFin, duración, costoTotal, y que el conductor recupere su disponibilidad.
- RFC1: Se verifica que el cliente tiene servicios registrados. Se hace un GET a /clientes/{id}/servicios y se revisa la respuesta del servidor. Se espera que se retorne una lista

ENTREGA #3

de servicios ordenados por fecha y que se incluyan tipoServicio, estado, costoTotal, kilometros, duracion.

- RFC2: Se verifica que existan conductores con historial de servicios. Se realiza un GET a /conductores/top20 y se valida la respuesta. Se espera que retorne una lista de máximo 20 conductores en orden descendiente por cantidad de servicios. Esta lista incluye nombre, calificacionPromedio y totalComisiones.
- RFC3: Se verifica que existan servicios con coordenadas o ciudad registrada. Se realiza un GET a /estadisticas/servicios?desde=YYYY-MM-DD&hasta=YYYY-MM-DD y se revisa la estructura. Se espera que para cada ciudad se retorne totalServicio, porcentajeUso.